



Carnegie Mellon University
Master of
Software Engineering

Program introduction and overview

LG Architecture Training Program

Paulo Merson

pleased
TO MEET YOU

Paulo Merson — pmerson@cmu.edu



**Carnegie
Mellon
University**

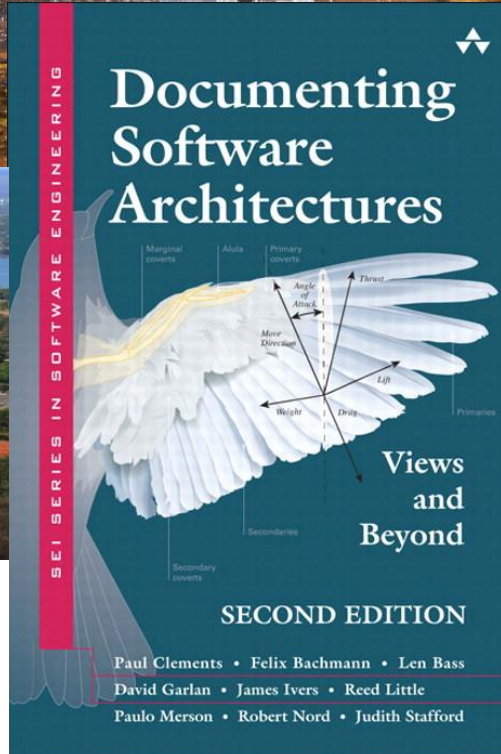


UnB



S3D

Software and Societal
Systems Department



Carnegie Mellon.

Software Engineering Institute

Agenda

➔ About CMU

About the LG-CMU program

Getting started

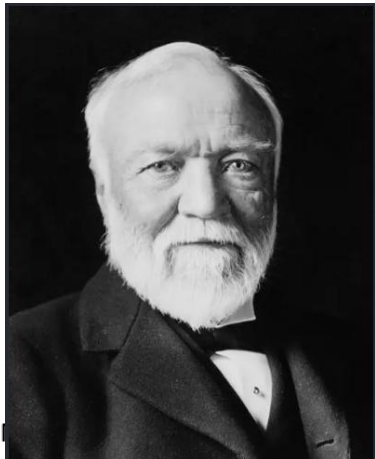
History of Carnegie Mellon University

- Founded by Andrew Carnegie in 1900
- Immigrant from Scotland, went from factory worker to richest man in the world
- Entrepreneur and industrialist, created technical schools for working-class men and women

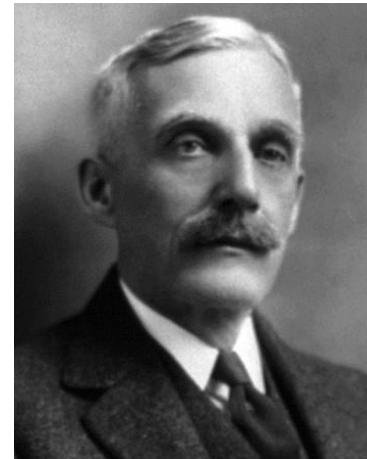


Carnegie Tech and then CMU

- School was *Carnegie Institute of Technology* (Carnegie Tech) until 1967 when a merger with the Mellon Institute created Carnegie Mellon University
- From the early days CMU had a philosophy of delivering higher education, while focusing on hands-on application of theory
- The first IBM computer arrived in 1956 and initiated a culture where information technology pervaded virtually all areas of study



Andrew Carnegie
1835-1919



Andrew Mellon
1855-1937

More history at
cmu.edu/about/history.html

CMU main campus location



Pennsylvania

- Other campuses:
 - Doha, Qatar
 - Mountain View, California (Silicon valley)

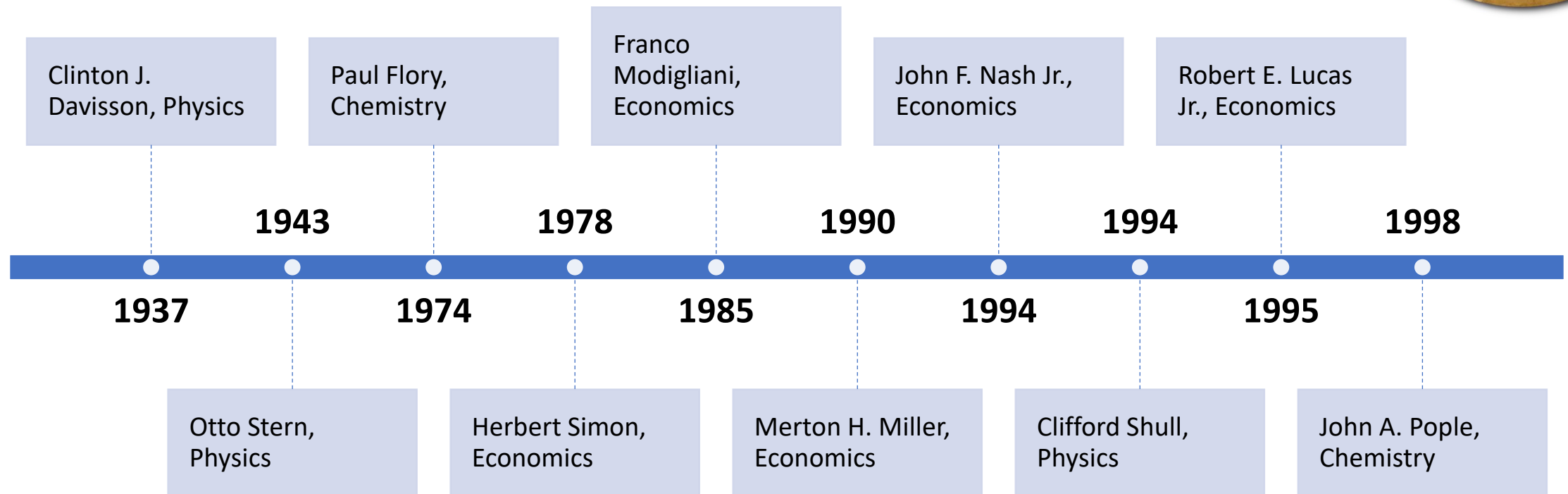


Pittsburgh

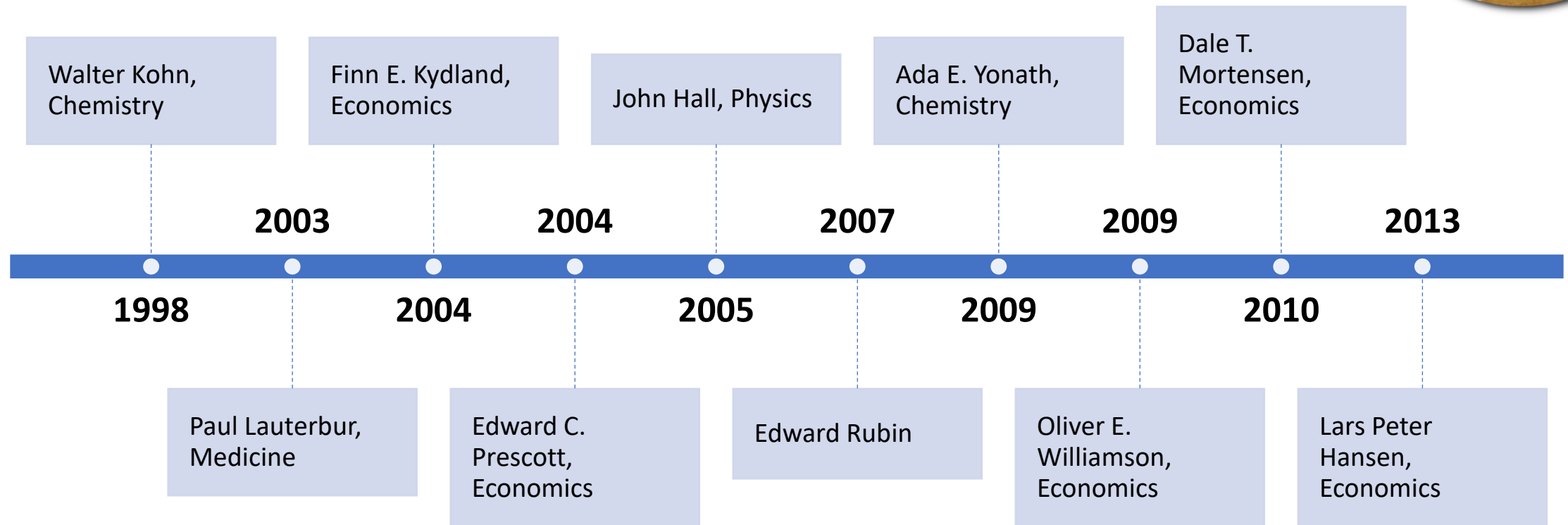




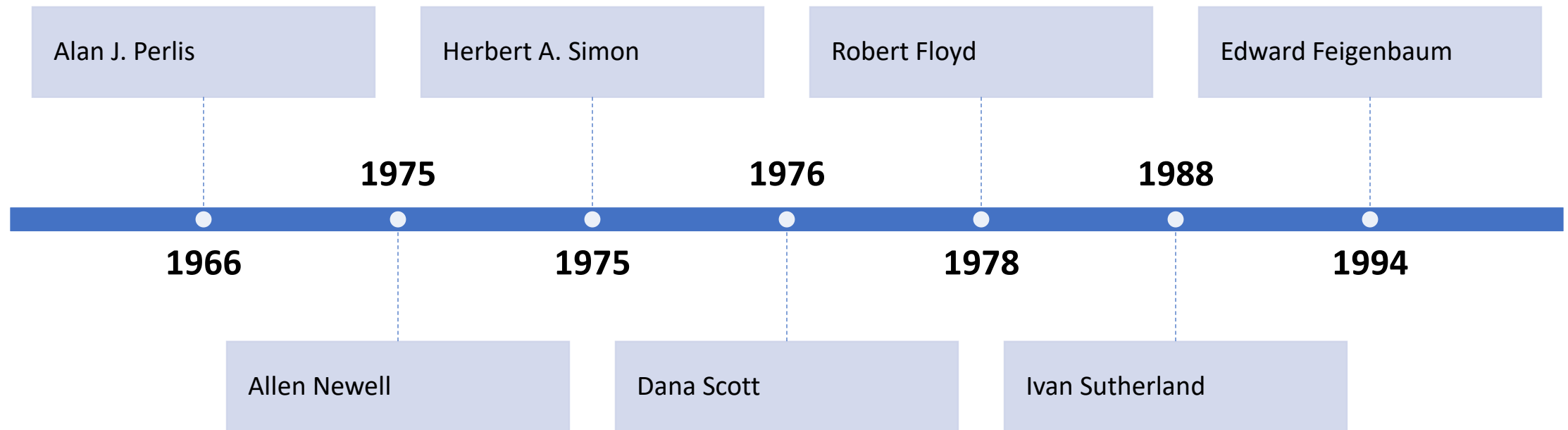
CMU Nobel Prize laureates (1)



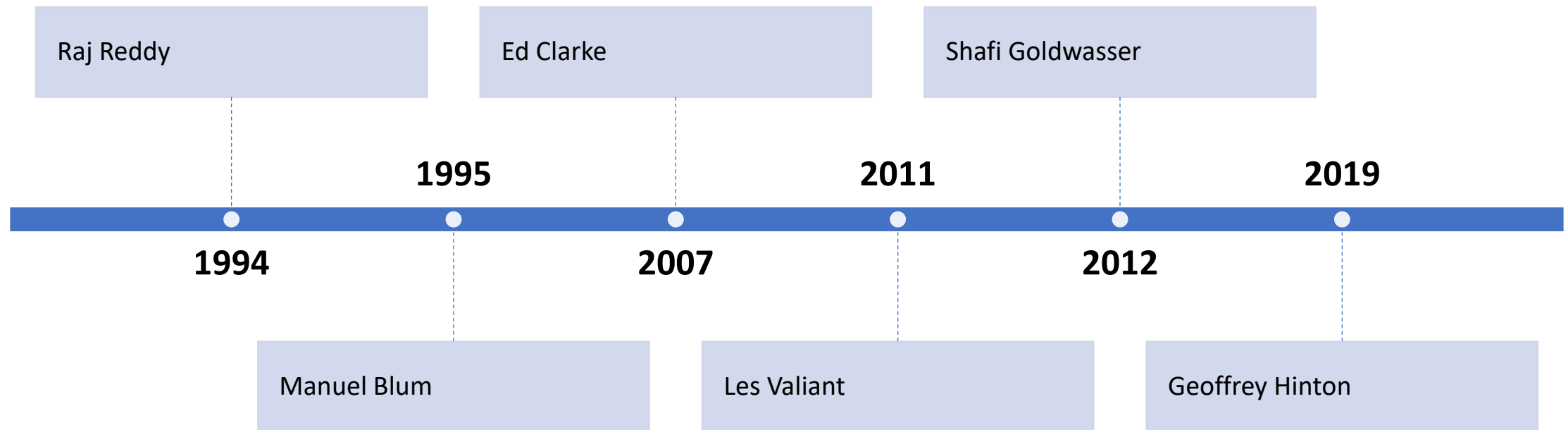
CMU Nobel Prize laureates (2)



CMU Turing Award winners (1)



CMU Turing Award winners (2)



Software Engineering at CMU ₍₁₎

- Department of Computer Science founded in 1965—one of the first such programs in the world
- Allan Newell ('57)—father of AI and inventor of the linked-list
- Dave Parnas (PhD '65)—developed the concept of information hiding
- Andy Bechtolsheim ('76) and Vinod Khosla ('78)—co-founders of Sun Microsystems ('82)
- Home of the Software Engineering Institute ('84)
- Randy Pausch (CS'88)—*The Last Lecture* author & creator of Alice educational software



Paulo Merson



3m5/3
Kbpsh



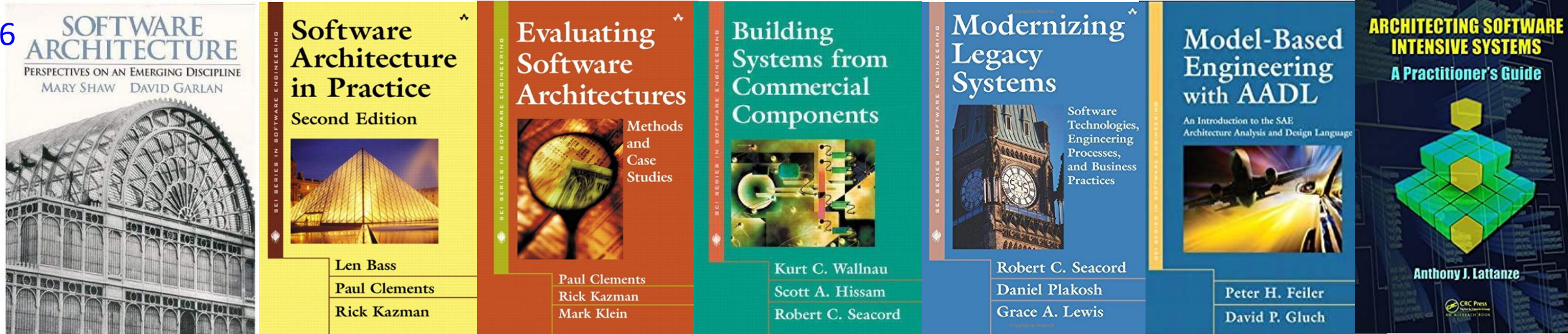
Software Engineering at CMU (2)

- Robotics Institute ('79)—first robotics program in the world
- James Gosling (PhD CS '83)—designer and creator of Java ('94)
- Michael Loren Mauldin (PhD CS '84)—creator of 1st modern commercial search engine, Lycos
- Watts Humphrey—“father of software quality” and creator of the capability maturity model ('86)
- Ed Clarke ('95)—co-developed model checking
- First Computer Emergency Response Team (CERT) established in 1988

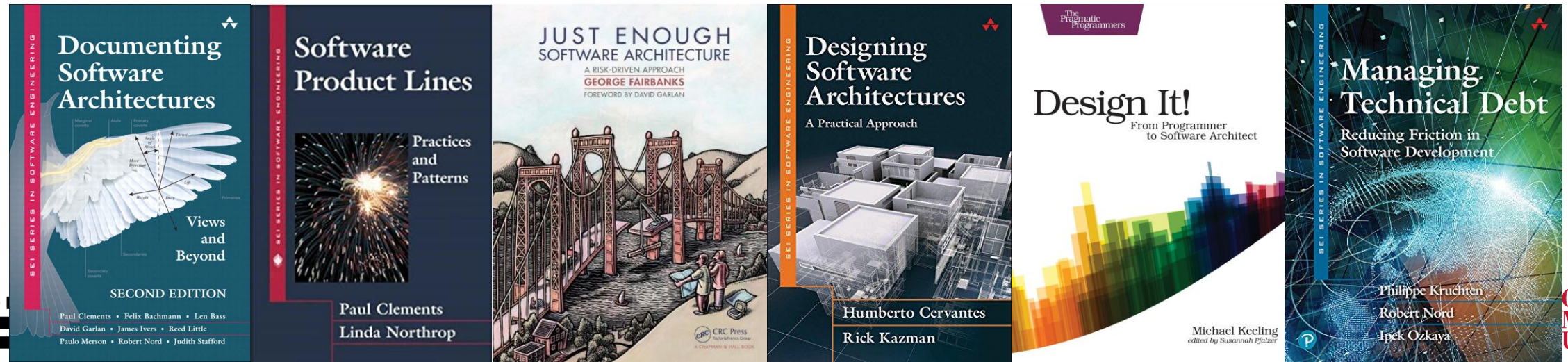


Software Architecture at CMU

1996



CMU has been at the forefront of software architecture since its beginning



2019

Agenda

About CMU

➔ About the LG-CMU program

Getting started

The LG-CMU Architecture Training Program

- The LG-CMU summer architecture training program has existed since 2006
- The program has evolved over the years
- The focus of the program since 2007 has been software architecture design
- Today the program is part of the LG architecture training program



The Program Focus Areas

- The program is structured around architecture analysis and design, but also involves process and project management
- The focus is software architecture, but we'll discuss ancillary related topics that support architecture concerns and activities
- This is a **very intense program** that has been tailored for LG to address specific training needs

Learning objectives

- Present fundamental architectural design principles
- Study various architectural structures, styles, and patterns, and understand tradeoffs in systemic properties
- Study techniques and methods for architecture analysis and design
- Explore ways to promote specific quality attribute properties



Architecture and more

- Our first focus will be architecture design principles, methods and techniques
- But we will also relate architecture to issues associated with
 - organizations and teams and the impact on architectures
 - project management and the relationship to architectures
 - analysis and testing techniques that support systemic design
- And we'll **learn by doing** through individual assignments and the team project

Important Course Themes



There is no recipe for design, design is a way of thinking



Software system design does not happen in isolation



Software design is a technical activity, but influences organizations, engineering practices, business and marketing



Design is dependent upon judgment, and good judgment comes from principles and experience



We are here to teach principles and best practices, not what you do at work

Course organization

- Five weeks
- Recorded lectures
- Reading assignments
- Individual homework assignments
- Individual weekly quizzes (exams)
- Team project with different milestones

You will be learning and right away applying what you learned 🧐

Weekly learning routine

- Specific lectures to watch (there are lectures to be watched before week 1 🤖)
- Texts to read (reading assignments)
- Weekly quiz
- Individual assignments
- Live online (synchronous) classes
- Meetings with project mentors

Individual assignment

- Referred to as A1, A2, etc.
- Will allow you to apply the ideas learned in the lectures
- These ideas and practices will also be useful for the team project
- These assignments will be posted on Canvas
- Your answers will be graded by the TA

*Ask questions or seek help if you are having trouble—
interaction with the instructors is highly encouraged*



Individual weekly quiz

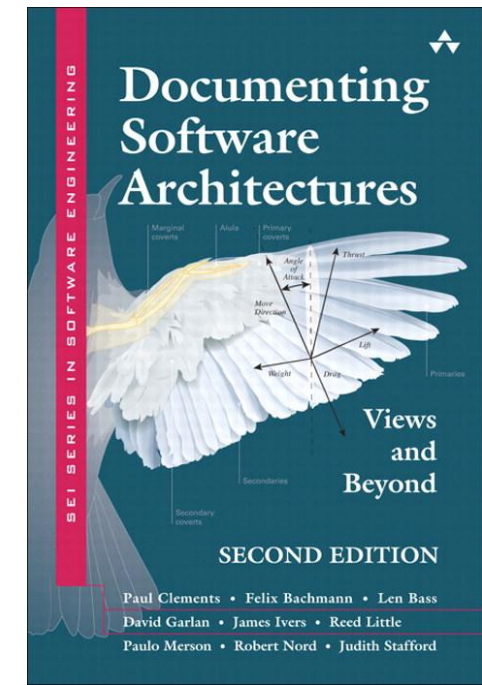
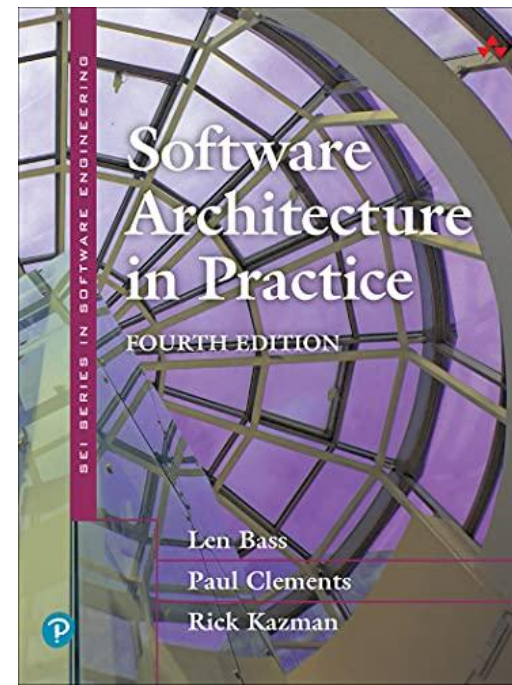
- Will also be done on canvas
- Grades and feedback will be visible the day after the deadline
- On canvas you'll see:
 - **Required reading** assignment: text you should read ideally *before* watching the lectures
 - **Weekly quiz**: to be done *after* watching the lectures and going through slides and readings
- Answer the questions based on reading assignments, slides, and lecture presentation

*There's a time limit (ex.: 60 min) so study **before** you start the quiz*

Textbooks

- [Bass21] L. Bass, P. Clements, R. Kazman. *Software Architecture in Practice*, 4th Edition. Addison-Wesley, 2021.
- [Clements10] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford. *Documenting Software Architectures: Views and Beyond*, Second Edition. Addison-Wesley, 2010.

*Ebooks can be purchased at
informit.com (discount code: BUY2)*



Live ~~online~~ in-person classes

- Opportunity to review questions in the previous quiz and assignment
- We can also discuss how to apply ideas in the course to your work at LG
- We may also play games 🐱



Team project

- Probably the most important element of the training
- There will be 3 milestones, including a demo in the last week
- Each group will have periodic meetings with an assigned mentor
- The team project is so important that we'll have other lectures just to present it 😊



Schedule of lectures (subject to change)

Week	Topics
Pre work & week 0	Introduction to the program; history of Software Architecture; Architecture motivation and definition; Architecture and Agile
1	Architecture drivers; quality attributes; performance, availability, security, usability, modifiability, and their tactics;
2	Architecture evaluation; architecture views; module views; layered; MVC; data model and domain model; runtime views; pipe and filter; REST; P2P; pub-sub
3	Deployment views; cloud computing; software interfaces; behavior diagrams; architecture documentation template; ADR; embedded architecture; IoT SE
4	Distributed systems; containerization; docker; microservices; NoSQL databases;
5	Product lines; adoption strategies; Architecture for ML Systems

See the syllabus on canvas for a more detailed and up-to-date schedule

Agenda

About CMU

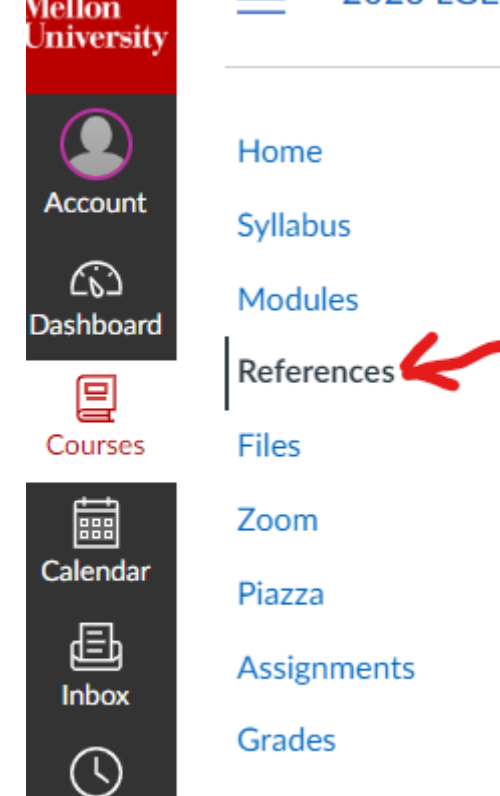
About the LG-CMU program

➔ Getting started

What else?

- References are in the syllabus document (direct link on Canvas)
- Try to connect concepts in the class to your practical experience
- Take notes when you are watching lectures

Please use Piazza for questions and discussion



Cheating Policy

- Cheating is not acceptable behavior!
- Examples of cheating include:
 - Asking former students for answers or artifacts
 - Copying work from other classmates
 - Using laptops or cell phones to share answers
- Instances of cheating will be reported to the LG program directors

Cooperation Policy

- We encourage you to discuss with other students anything that is presented throughout the week
- We encourage you to study together and share ideas and discussion
- Quizzes and individual assignments must be your work only!
- Team project tasks: share everything and work together on everything

Get Ready!

- This program is a lot of work, but we think it will be well worth your effort
 - Try to stay ahead of the workload and proactively manage your time
 - Your life will be miserable if you do not manage your time
- Focus on teamwork and communication
 - Discuss, compromise, work together, delegate, keep your commitments, and keep it professional
 - Your life will be miserable if you can't work as a team

I recommend...

*Good judgment comes from
experience
and experience comes from bad
judgment.*

—Fred Brooks



Increase your learning experience

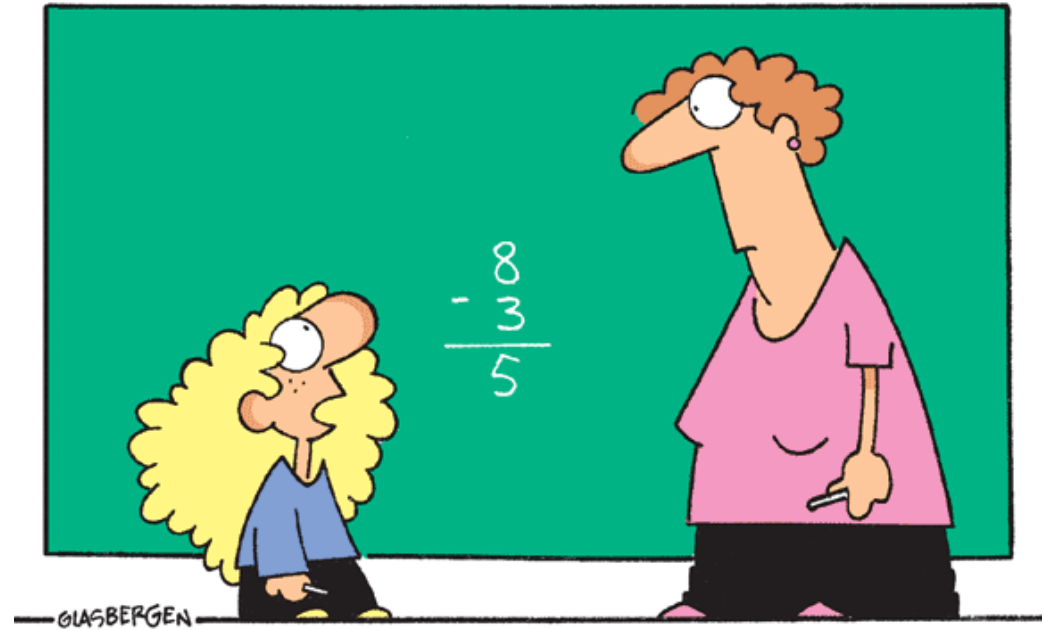
- Volunteer to perform the tasks you're less familiar with
- Increase your knowledge in software architecture, but also improve your:
 - Oral and written communication skills (also in English)
 - Management abilities
 - Leadership skills
 - Interpersonal skills



Be curious, be inquisitive

- Participate in class and on Piazza
- If you disagree with an idea, feel free to challenge it, to question *why*
- Provoke thought and reflection

© Randy Glasbergen. www.glasbergen.com



**“What gives anyone the right to take 3 away from 8?
Why should 8 be diminished to the lesser value of
5 just to satisfy someone’s obsession with math?”**

For the more experienced developers

- Check in your ego at the door
- Be open minded
- Try new techniques, tools, methods



Learn from mistakes

- Don't be afraid to make mistakes... but have the courage to discover why
- *If you succeed but you don't know why, you failed*
- *If you failed but you know why, you succeeded*



Work hard

- Work hard and work smart
- Be proactive
- Details matter
 - You're being observed
 - Don't settle for good



Not my job attitude 🤪

Enjoy the experience

- Time management is critical to success and sanity
- Make time for yourself and get away from the computer





Questions? now or later

Paulo Merson
pmerson@cmu.edu